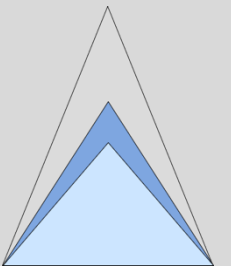
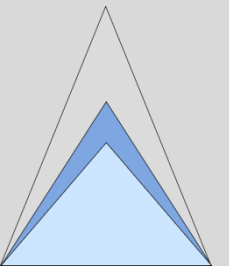


# Introduction to the Apheleia Verification Library

The Python Based Alternative to UVM for scalable, re-usable test-bench designs

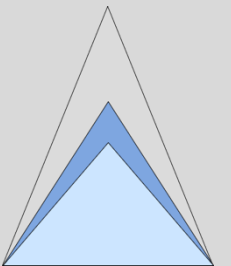


# Overview



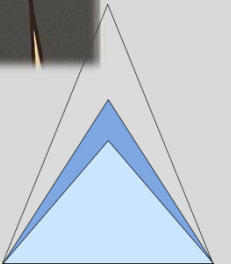
# Overview – What is AVL

- The Apheleia Verification Library (AVL) is an open-source python library
- AVL is not a UVM implementation in python
- AVL is not a minimal test-bench language
- AVL takes combines the re-use best practices of UVM and efficiency of python
- AVL is an engineer driven test-bench library enabling scalable verification environments with a focus on productivity – not methodology



# Overview – Who is AVL Aimed At?

- Novices and students
- Industry experts
- Hobbyists
- Professionals
- Anyone who wants to spend more time doing verification and less time developing code



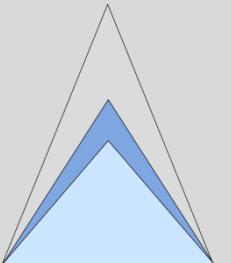


## Pros

- Successful at bringing standardization to the verification industry
- Strong methodology
- Wide range of available VIP
- Well understood terminology

## Cons

- Limited to major EDA vendors with implementation specific version
- Inconsistent views on best practices
- Code intensive development



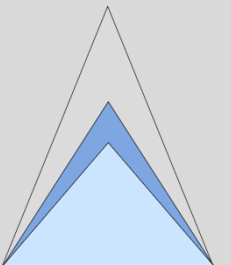


## Pros

- 100% Open-Source
- Universal simulator support
- Active User Community
- Near Zero compile time
- Widely used and understood language
- Rich and diverse range of useful libraries

## Cons

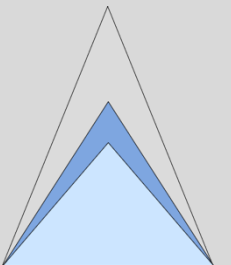
- No common methodology
- Limited available VIP
- Software centric
- Different





# AVL Features

- HDL centric variables
- Constrained Random
- Familiar methodology
  - Sequences
  - Drivers
  - Agents
- Familiar re-use
  - Factory
  - Phases
  - TLM style ports
- Functional Coverage
  - Run-time defined
- Statistical Coverage
- Visualization
- Multi-purpose logging
  - Human Readable
  - Machine Readable
- Searchable Trace



# AVL Variables

| System Verilog | Python     | AVL                                |
|----------------|------------|------------------------------------|
| shortint       | int        | avl.Int16                          |
| int / integer  | int        | avl.Int32                          |
| longint        | int        | avl.Int64                          |
| byte           | int        | avl.Byte / avl.Int8                |
| logic / bit    | bool / int | avl.Logic / avl.Bool / avl.Uint<N> |
| time           | int        | avl.Int64                          |
| real           | float      | avl.Double / avl.Fp64              |
|                | float      | avl.Half / avl.Fp16                |
| shortreal      | float      | avl.Float / avl.Fp32               |
| string         | str        | str                                |
| enum           | Enum       | avl.Enum                           |



# AVL Variables

- Once defined all AVL variables behave like python variables
  - Arithmetic operations
  - Comparison
- Wrapping and sign are handled naturally
- Each variable can have a defined string format for easier debug

```
a = avl.Uint(250, width=8)
b = avl.Uint(10, width=8)

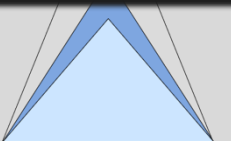
assert a + b == 4           # 250 + 10 = 260 % 256 = 4
c = avl.Uint(a, width=8)
c += b
assert c == 4               # 250 + 10 = 260 % 256 = 4

assert a - b == 240         # 250 - 10 = 240
assert b - a == 16          # 10 - 250 = -240 % 256 = 16
c = avl.Uint(b, width=8)
c -= a
assert c == 16              # 10 - 250 = -240 % 256 = 16

assert a * b == 196         # 250 * 10 = 2500 % 256 = 196
c = avl.Uint(a, width=8)
c *= b
assert c == 196             # 250 * 10 = 2500 % 256 = 196

assert a // b == 25         # 250 // 10 = 25
c = avl.Uint(a, width=8)
c //= b
assert c == 25              # 250 // 10 = 25

assert a % b == 0           # 250 % 10 = 0
c = avl.Uint(a, width=8)
c %= b
assert c == 0               # 250 % 10 = 0
```

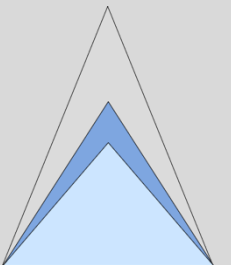


# AVL Variables

- SystemVerilog like bit-slicing supported

```
# slices
c = avl.Logic(0xdeadbeef, width=32)
assert c[0:8] == 0xef
assert c[24:32] == 0xde
assert c[8:24] == 0xadbe
assert c[19] == 1
assert c[14] == 0

c[8:16] = 0xab
assert c == 0xdeadabef
c[3] = 0
assert c == 0xdeadabe7
```



# AVL Variables

- Native floating-point values based on NumPy
- Helper functions to interact with hardware

```
a = avl.Fp16(2.0)
b = avl.Fp16(3.0)

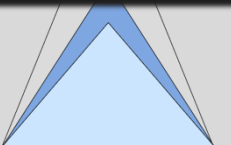
# Arithmetic
assert a + b == 5.0          # 2.0 + 3.0 = 5.0
c = avl.Fp16(a)
c += b
assert c == 5.0             # 2.0 + 3.0 = 5.0

assert a - b == -1.0        # 2.0 - 3.0 = -1.0
c = avl.Fp16(a)
c -= b
assert c == -1.0           # 2.0 - 3.0 = -1.0

assert a * b == 6.0         # 2.0 * 3.0 = 6.0
c = avl.Fp16(a)
c *= b
assert c == 6.0            # 2.0 * 3.0 = 6.0

assert b / a == 1.5         # 3.0 / 2.0 = 1.5
c = avl.Fp16(b)
c /= a
assert c == 1.5            # 3.0 / 2.0 = 1.5

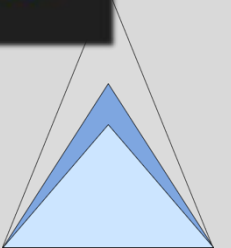
# Power
assert a ** 2 == 4.0        # 2.0 ** 2 = 4.0
c = avl.Fp16(a)
c **= 2
assert c == 4.0            # 2.0 ** 2 = 4.0
```



# AVL Variables

- Structures
  - Verilator and some other simulators flattens structs
  - Helper class provided to be simulator agnostic

```
class packed_struct_t(avl.Struct):  
    single_bit : avl.Bool = avl.Bool(False)  
    multi_bit : avl.Uint32 = avl.Uint32(0)  
    state_enum : avl.Enum = avl.Enum("S0", {"S0" : 0, "S1" : 1, "S2" : 2})
```



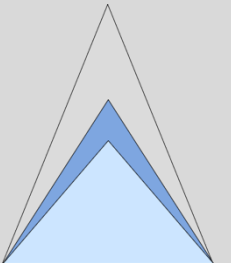
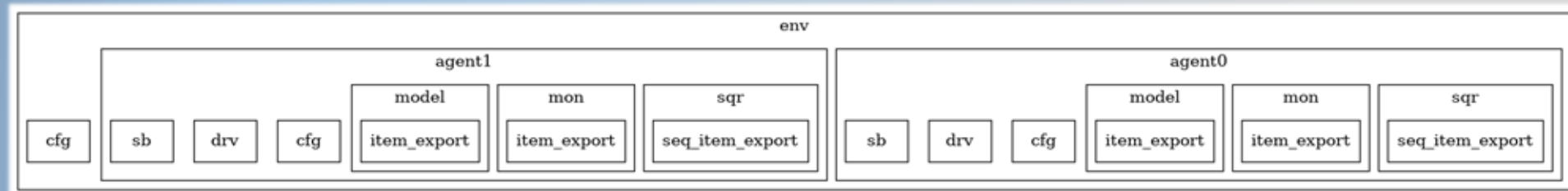
# Logging

- AVL logging sits on-top of CocoTB logging
- Allows log messages to be exported into any Pandas supported file format
  - csv, json, yml, markdown, rst, text
- Human readable and machine readable
- Trivial to group, sort, filter and prioritize messages and errors using Pandas SQL style conditions
- All implemented in simple class
  - Messages can be modified / translated using common python libraries like Google Translate



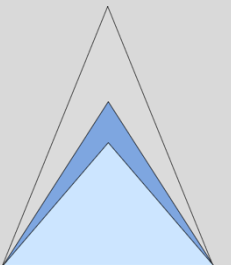
# Visualization

- Simple Testbench Visualization
  - Tree View
  - Diagram



# Constrained Random

- UVM benefits from the constrained random features of SystemVerilog
- Python random supports randomization, but lacks constraints
- There are many theorem solvers available in Python
  - Each has benefits and limitations
  - Can be confusing to decide on best approach



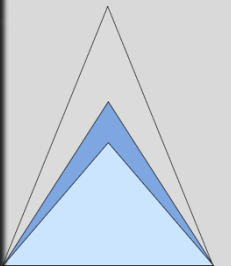


# Constrained Random

- AVL utilizes [Z3](#) – an open-source theorem prover from Microsoft
- Supports bool, int, uint, enum and **float** numbers and wide variables
- Supports hard and soft constraints
- Well documented and well maintained

```
self.a = avl.Logic(0, width=8, fmt=hex)
self.b = avl.Logic(0, width=8, fmt=hex)

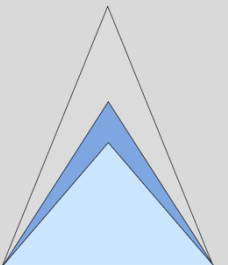
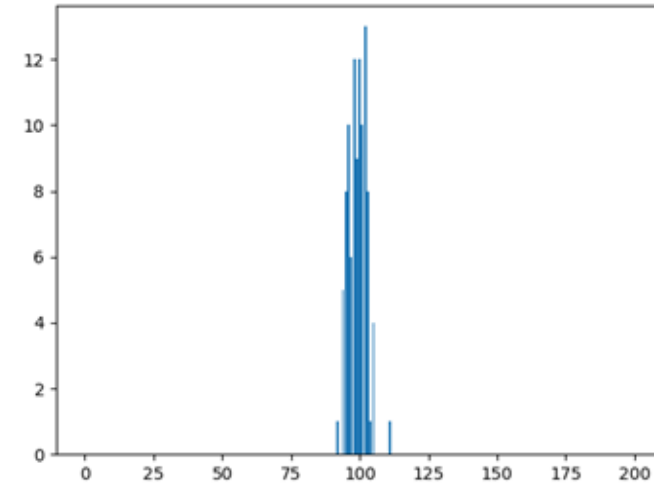
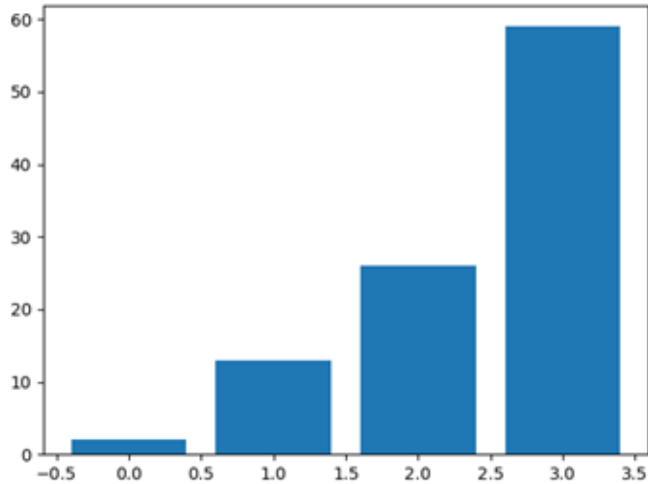
self.add_constraint("c_0", lambda x: Or(x == 0, x == 100), self.a)
self.add_constraint("c_1", lambda x: And(x >= 5, x <= 100), self.b)
self.add_constraint("c_2", lambda x, y: Implies(x == 0, y == 10), self.a, self.b)
```



# Distributions

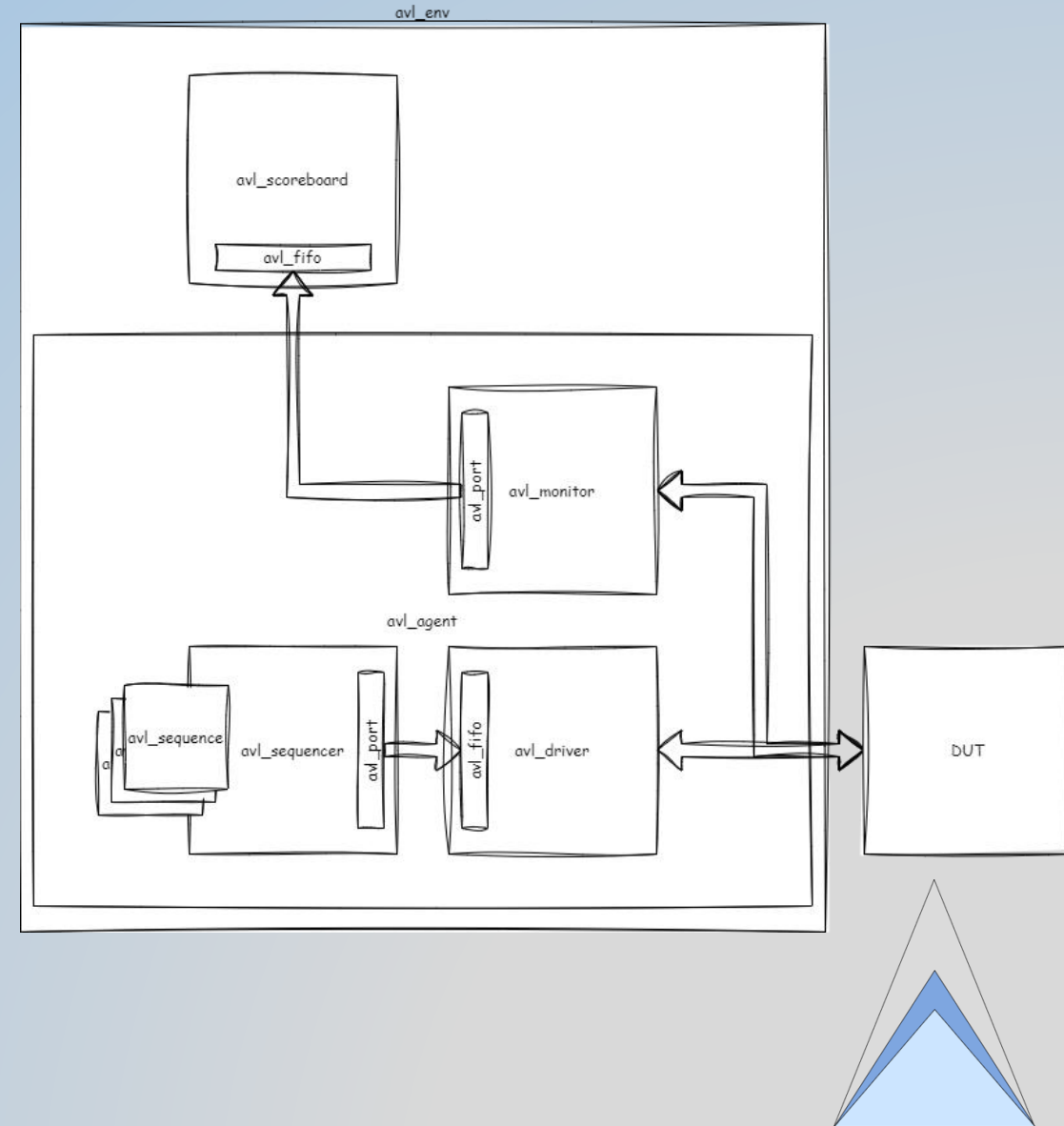
```
self.a = avl.Logic("a", 0, width=8, fmt=hex)
self.a.add_constraint(
    "d_0", lambda x: x == random.choices([0, 1, 2, 3], k=1, weights=[1, 2, 4, 8])[0]
)

self.b = avl.Logic("b", 0, width=32, fmt=hex)
self.b.add_constraint("d_1", lambda x: x == int(np.random.normal(100, 3)))
```



# AVL Methodology

- AVL follows the UVM methodology
- Familiar and consistent
- Terminology and behaviour maintained where appropriate
- No need for parameterization
- Direct access
  - No requirement for virtual interfaces



# Factory

- UVM factory was a fudge due to language limitation
- AVL factory built in natively
- User extendable specificity function to decide override precedence

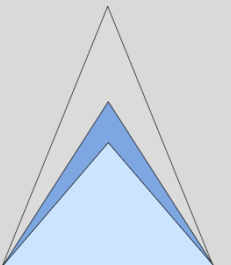
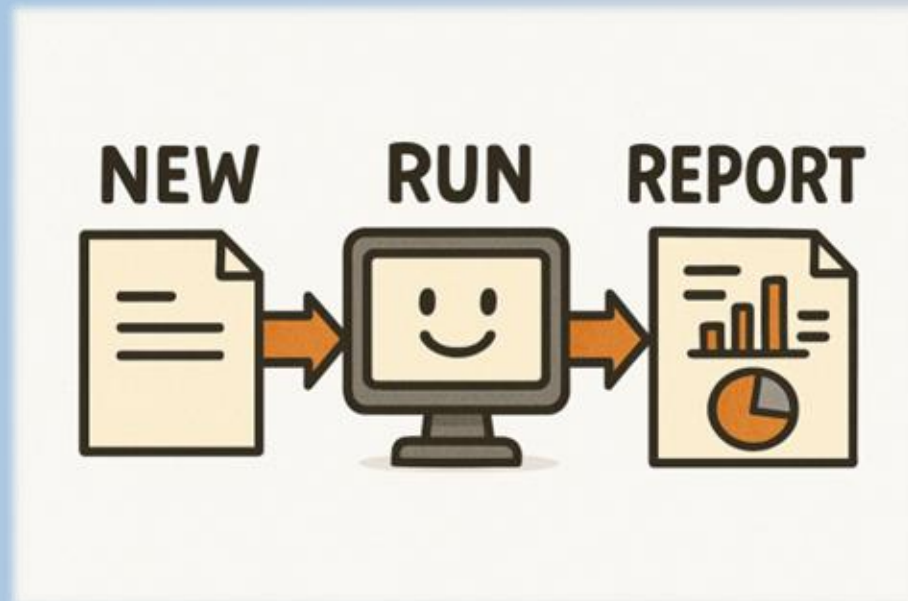
```
avl_factory.set_override_by_instance('env.s', sub_comp_B)  
avl_factory.set_override_by_instance('env.o', object_B)  
  
e = example_env('env', None)
```

```
avl_factory.set_variable('env.a', 100)  
  
e = example_env('env', None)
```



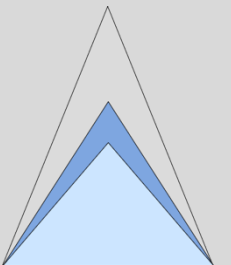
# Phases

- Phases are useful, but the need for them varies based on the type of testbench
- AVL supports adding, inserting and removing phase, but by default only provides 2 – Run & Report



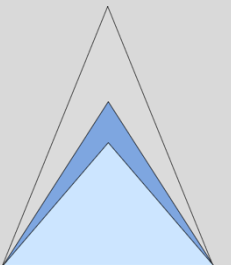
# Scoreboards

- Common mistakes weaken scoreboards
- Index scoreboard provides same API, but splits items into multiple in-order scoreboards based on an item attribute
- All scoreboards provide:
  - End-of-sim empty checks
  - Minimum check count



# Testbench Helper Functions

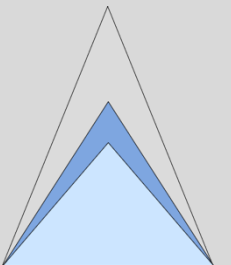
- Environment class provides common testbench helper functions
  - Clock Generator defined by frequency
  - Sync / Async reset generators
  - Hard and Soft Timeouts
  - Tickers
- Vanilla template provided
  - Only need to override item type and implement driver / monitor run\_phase





# Ports

- Only 1 port always capable of broadcast
- `avl.List` extends native python list
  - Adds `blocking_get()` / `blocking_pop()`
- `avl.Fifo` extends `avl.List`
  - Adds `blocking_push()`
- Callbacks built into `avl.Transaction` not sequencer
  - Synchronisation on item-by-item
  - Simplifies sequences with multiple in-flight transactions or out-of-order responses

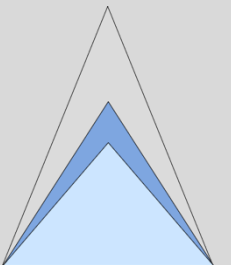


# Functional Coverage

- AVL functional coverage familiar to SystemVerilog coverage
  - Covergroups, Coverpoints, Covercrosses
- Defined at run-time
- Flexible bin definition
  - Value in list, range or function
- Analysed and exported using Pandas
  - Simple, multi-sim merging and ranking

```
self.cg = avl.Covergroup("cg", self)
self.cp_a = self.cg.add_coverpoint("cp_a", lambda: self.a)
self.cp_a.add_bin("bin", 1, 2, range(100, 200))

self.cp_b = self.cg.add_coverpoint("cp_b", lambda: self.b)
self.cp_b.add_bin("bin", lambda x: x == 10)
```



# Functional Coverage – Statistical Bins

- Bins can also collect statistics
  - Not just hit / unhit
  - Min, Max, Mean and StdDev of sampled values
- Provides native support for performance measurements

Coverage

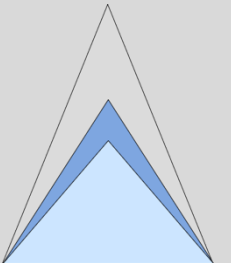
Show 100 entries

Search

| covergroup        | name        | bin        | at_least        | count        | min        | max        | mean        | variance        | stddev        |
|-------------------|-------------|------------|-----------------|--------------|------------|------------|-------------|-----------------|---------------|
| Filter covergroup | Filter name | Filter bin | Filter at_least | Filter count | Filter min | Filter max | Filter mean | Filter variance | Filter stddev |
| CG                | CP_A        | bin[0]     | 1               | 0            |            |            |             |                 |               |
| CG                | CP_A        | bin[1]     | 1               | 0            |            |            |             |                 |               |
| CG                | CP_A        | bin[2]     | 1               | 2            |            |            |             |                 |               |
| CG                | CP_A        | bin[3]     | 1               | 1            |            |            |             |                 |               |
| CG                | CP_A        | bin[4]     | 1               | 3            |            |            |             |                 |               |
| CG                | CP_A        | bin[5]     | 1               | 2            |            |            |             |                 |               |
| CG                | CP_A        | bin[6]     | 1               | 0            |            |            |             |                 |               |
| CG                | CP_A        | bin[7]     | 1               | 1            |            |            |             |                 |               |
| CG                | CP_A        | bin[8]     | 1               | 1            |            |            |             |                 |               |
| CG                | CP_A        | bin[9]     | 1               | 0            |            |            |             |                 |               |
| CG                | CP_B        | bin        | 1               | 10           | 2.0        | 8.0        | 4.4         | 3.822222        | 1.95505       |

Showing 1 to 11 of 11 entries

Previous 1 Next



# Functional Coverage – Reporting

- Simple coverage reporting
  - Coverage Merging
  - Coverage Ranking
  - Regression optimization

**Coverage Report**

Show 10 entries Search:

| name       | total bins | covered bins | coverage |
|------------|------------|--------------|----------|
| coverage_0 | 40.0       | 14.0         | 35.0     |
| coverage_1 | 40.0       | 9.0          | 22.5     |
| coverage_2 | 40.0       | 14.0         | 35.0     |
| coverage_3 | 40.0       | 4.0          | 10.0     |
| coverage_4 | 40.0       | 11.0         | 27.5     |
| coverage_5 | 40.0       | 14.0         | 35.0     |
| coverage_6 | 40.0       | 2.0          | 5.0      |
| coverage_7 | 40.0       | 9.0          | 22.5     |

Showing 1 to 8 of 8 entries Previous 1 Next

**Merged Coverage Report**

Show 10 entries Search:

| name            | total bins | covered bins | coverage |
|-----------------|------------|--------------|----------|
| Merged Coverage | 40         | 37           | 92.5     |

Showing 1 to 1 of 1 entries Previous 1 Next

**Ranked Coverage Report**

- Ranked Coverage
- Merged Coverage (By Contributor)

**Ranked Coverage Report**

Show 10 entries Search:

| name            | score | unique runs | rare runs | total runs |
|-----------------|-------|-------------|-----------|------------|
| coverage_2.json | 4100  | 4           | 10        | 14         |
| coverage_0.json | 2120  | 2           | 12        | 14         |
| coverage_7.json | 2070  | 2           | 7         | 9          |
| coverage_5.json | 1130  | 1           | 13        | 14         |
| coverage_4.json | 1100  | 1           | 10        | 11         |
| coverage_1.json | 1080  | 1           | 8         | 9          |
| coverage_6.json | 1010  | 1           | 1         | 2          |
| coverage_3.json | 40    | 0           | 4         | 4          |

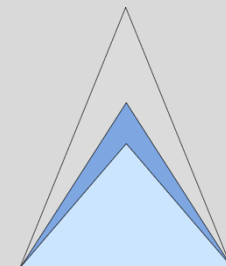
Showing 1 to 8 of 8 entries Previous 1 Next

**Merged Coverage Report (By Contributor)**

Show 10 entries Search:

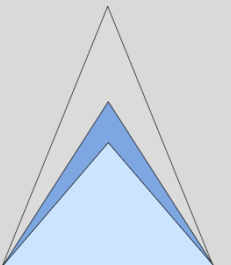
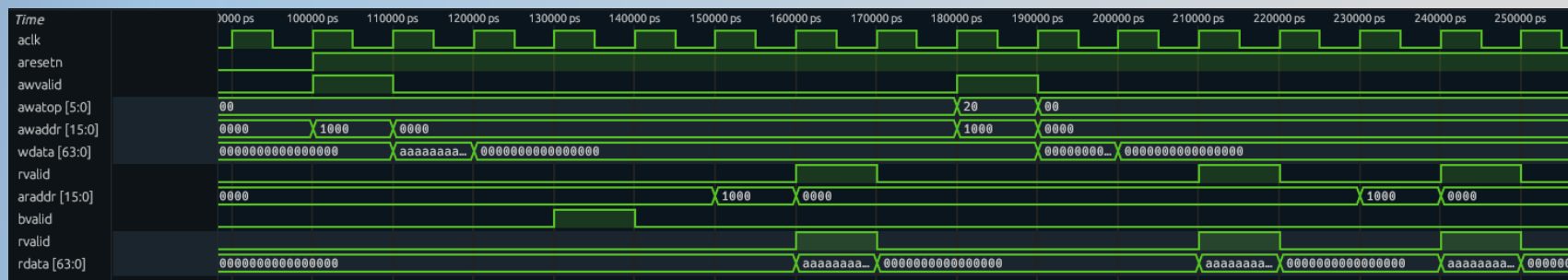
| covergroup | name | bin | contributor                                         |
|------------|------|-----|-----------------------------------------------------|
| cg         | cp_a | a0  | [coverage_0.json, coverage_3.json]                  |
| cg         | cp_a | a1  | [coverage_4.json, coverage_5.json]                  |
| cg         | cp_a | a10 | [coverage_2.json]                                   |
| cg         | cp_a | a11 | [coverage_4.json]                                   |
| cg         | cp_a | a12 | [coverage_5.json]                                   |
| cg         | cp_a | a13 | [coverage_0.json, coverage_2.json, coverage_7.json] |
| cg         | cp_a | a14 | [coverage_2.json, coverage_5.json]                  |
| cg         | cp_a | a15 | [coverage_2.json]                                   |
| cg         | cp_a | a16 | [coverage_1.json, coverage_6.json]                  |
| cg         | cp_a | a17 | [coverage_0.json, coverage_2.json, coverage_4.json] |

Showing 1 to 10 of 37 entries Previous 1 2 3 4 Next



# VIP

- A key principle of AVL is the ability to create re-usable Verification IP
- AMBA has been the first focus for this effort
  - [AXI-STREAM](#)
  - [APB](#)
  - [AXI](#)



# Documentation

- High quality documentation
  - <https://avl-core.readthedocs.io/en/latest/index.html>
- Multiple Examples
  - <https://github.com/projectapheleia/avl/tree/main/examples>

avl

latest

Search docs

CONTENTS:

- Welcome to AVL's Documentation
- Quickstart Guide
- AVL Logging
- AVL Variables
- AVL Base classes
- AVL Synchronization and TLM
- AVL Factory
- AVL Phases
- AVL Methodology
- AVL Constraints
- AVL Trace
- AVL Coverage
- AVL Coverage Reports
- AVL Templates
- AVL Visualization
- All Modules

Develop and launch modern apps with MongoDB Atlas, a resilient data platform.

Ad by Ethicalads

Apheleia Verification Library Documentation

View page source

Apheleia Verification Library Documentation

Contents:

- Welcome to AVL's Documentation
- Quickstart Guide
- AVL Logging
- AVL Variables
- AVL Base classes
- AVL Synchronization and TLM
- AVL Factory
- AVL Phases
- AVL Methodology
- AVL Constraints
- AVL Trace
- AVL Coverage
- AVL Coverage Reports
- AVL Templates
- AVL Visualization
- All Modules

Additional Libraries / UVCs

- avl-apb - AMBA APB UVC for Apheleia Verification Library
- avl-axi-stream - AMBA AXI-STREAM UVC for Apheleia Verification Library
- avl-axi - AMBA AXI5 UVC for Apheleia Verification Library

Indices and tables

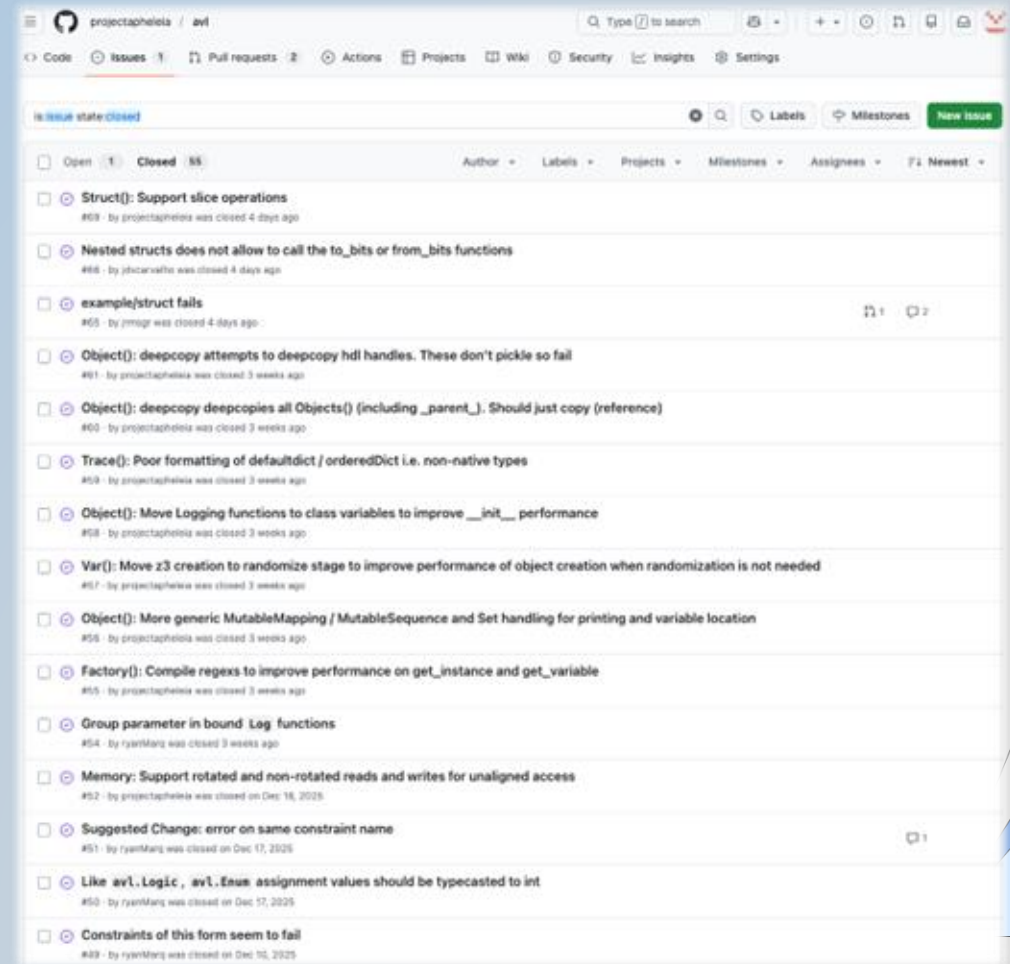
- Index
- Module Index
- Search Page

Next

latest

# Issues and Releases

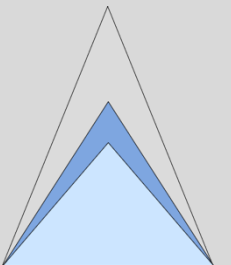
- Regular bug fixes and release
- Community submitted PRs



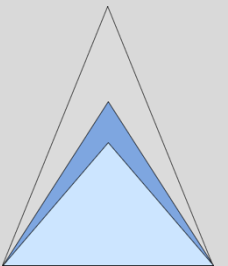


# Coming Soon

- RISC-V Functional Coverage Framework
  - Collect functional coverage from [SPIKE](#)
  - Simple / extensible framework
  - No need to execute on RTL to know your ISA coverage
- QEMU integration
  - Shift Left – Software style testing on Hardware test-benches
- RAL
  - Register Abstraction Layer
  - Simpler, more user-friendly version than UVM-RAL



# Questions and Answers



# Thank You

<https://github.com/projectapheleia/avl>

<https://avl-core.readthedocs.io/en/latest/index.html>

